

Katana Pay — Gateway Integration Kit

Hosted-checkout UPI pay-ins for any platform · Base URL `https://katanapay.co` · API v1

What you'll do: your server creates a **signed order** → you redirect the customer to the **Katana Pay hosted payment page** → the customer pays by UPI → Katana **POSTs a signed status callback** to your server *and* redirects the customer back to your `return_url` . You verify every signature with the **Key + Salt** issued to your merchant account.

Contents

- 1. Credentials (Key + Salt)
- 2. Endpoints
- 3. Create an order
- 4. Sign the request
- 5. Redirect to the payment page
- 6. Receive the status callback
- 7. Status enquiry (and long-poll)
- 8. Status & response codes
- 9. Traceability — following one payment end to end
- 10. Sample code (PHP / Node / Python)
- 11. Test mode
- 12. Go-live checklist

1. Credentials (Key + Salt)

Every merchant account gets its **own dedicated Key** and **Salt**, issued from the Katana dashboard (**Integration** → **Generate**, or ask your Katana account manager). They are not shared between merchants, and every order and callback is bound to the pair that signed it.

Test and live pairs

Pair	Key starts with	Orders it creates

Test	<code>mk_test_</code>	Test orders: they pay a sandbox UPI ID, never move real money and never appear in live totals, statements or settlement (§11).
Live	<code>mk_live_</code>	Real payments.

The Key you sign with decides the mode; there is no mode field to send. Keys issued before test mode (`mk_` followed by hex) are live and keep working unchanged.

Field	Meaning
Key	Public-ish identifier (<code>mk_test_...</code> / <code>mk_live_...</code>) sent with every request. Identifies the merchant and the mode.
Salt	Secret. Shown only once at issue. Used to sign your requests and to verify our callbacks. Never expose it in client-side code.
Scheme	Signing algorithm for your account: <code>HMAC_SHA256</code> . Every new pair is issued with it; pairs issued earlier with the legacy SHA-512 format (§4) keep working until regenerated.

Keep each Salt safe. If lost, regenerate that pair — which **invalidates only that pair** (the other mode is untouched), so update your server immediately. One active Key per mode.

2. Endpoints

Purpose	Method	URL
Create order (server-to-server)	POST	<code>https://katanapay.co/api/v1/katana-pay/order</code>
Hosted payment page (browser)	GET	<code>https://katanapay.co/pay/{order_id}</code>
Status enquiry	GET	<code>https://katanapay.co/api/pay-status/{order_id}</code>

Your Key + Salt is the only credential on these endpoints. Treat Salt disclosure as a full compromise of your integration: regenerate immediately and reconcile every order created in the interim.

Pay-in flows: P2P and Intent

Katana takes pay-ins on two flows, and your account is set up for **P2P**, **Intent** or **Both**:

Flow	How the customer pays	Order API	Status
------	-----------------------	-----------	--------

P2P	A UPI link or QR that pays the receiving UPI ID directly. Confirmed by the bank credit.	POST <code>/api/v1/p2p/order</code>	GET <code>/api/v1/p2p/order/{id}</code>
Intent	The payment is issued and confirmed by the payment gateway.	POST <code>/api/v1/intent/order</code>	GET <code>/api/v1/intent/order/{id}</code>

All three order APIs take the **same request**, the **same signature** and return the **same response**. The general `/api/v1/katana-pay/order` uses the flow selected for your account, so an existing integration keeps working unchanged. Call a flow's own API when you want that flow by name. The response says which flow the order took in "flow" (P2P or INTENT).

A flow your account is not set up for is refused with HTTP 409 and a code : FLOW_NOT_ENABLED (your account is on the other flow), FLOW_NOT_SELECTED (no flow has been selected yet) or FLOW_NOT_READY (the flow is selected but its setup is not complete). No order is created.

The flow status APIs accept the order id or the flow reference (P2P-000000123 / INT-000000123) and add flow and that reference to the usual status fields.

3. Create an order

POST `/api/v1/katana-pay/order` with a JSON body (form-encoded is also accepted). **Idempotent on txnid** — re-sending the same txnid returns the existing order rather than creating a second one, so a retry after a network timeout is always safe. A txnid is unique per mode: the same reference sent with a test Key and a live Key creates two separate orders.

Request fields

Field	Req	Description
key	yes	Your Key.
txnid	yes	Your unique order reference (≤ 60 chars). This is your handle on the payment forever — see §9.
amount	yes	Amount in rupees, e.g. "100" or "100.50" .
hash	yes	Signature over the order (see §4).
productinfo	no	Order description (part of the signature).
firstname	no	Customer name (part of the legacy SHA-512 signature).

email	no	Customer email (part of the signature).
phone	no	Customer phone.
return_url	no	Browser redirect target after payment. Overrides your saved default.
notify_url	no	Server-to-server status-callback target. Overrides your saved webhook URL.
currency	no	Defaults to <code>INR</code> .
mode	no	<code>QR</code> or <code>INTENT</code> .

`return_url` and `notify_url` must be **http(s)** URLs. Other schemes are rejected.

Response (HTTP 201 new / 200 reused)

```
{
  "verified": true,
  "merchant": "UK-108",
  "livemode": true,                                // false = created with a test Key
  "order": { "id": "8d20c0b6-...", "order_id": "ORDER-1001", "amount": 100, "status": "PENDING",
  ... },
  "pay_url": "https://katanapay.co/pay/8d20c0b6-...", // redirect the customer here
  "deeplinks": { "upi": "upi://pay?...", ... },
  "upi_intent": "upi://pay?...",
  "qr_payload": "upi://pay?..."
}
```

Take `pay_url` and send the browser there. If you are building your own payment screen instead, use `qr_payload` (render as a QR) or `upi_intent` (an app deeplink).

Merchants paid on a gateway's payment page

Some merchants collect through a gateway that takes the payment on its own secure page. Their orders have **no UPI link**: `deeplinks`, `upi_intent` and `qr_payload` are `null`, and one field is added:

```
{
  ...,
  "pay_url": "https://katanapay.co/pay/8d20c0b6-...", // still the page to send the customer
  to
  "deeplinks": null, "upi_intent": null, "qr_payload": null,
  "gateway_url": "https://katanapay.co/pay/8d20c0b6-.../go" // straight to the gateway's page,
  if you'd rather skip ours
}
```

Send the customer to `pay_url` as usual — it shows the order and hands over to the gateway — or straight to `gateway_url`. The result reaches you the same way (callback, status enquiry). The gateway's own limits apply, e.g. a minimum amount per payment; an order outside them is refused with `502` and the gateway's reason.

Error	Meaning
<code>401 invalid key</code>	Unknown Key.
<code>401 signature mismatch</code>	Hash didn't match — check your signing string.
<code>403</code>	Account blocked or suspended (<code>code</code> <code>MERCHANT_BLOCKED</code> / <code>MERCHANT_SUSPENDED</code>), or set up for payouts only (<code>code</code> <code>PAYIN_NOT_ENABLED</code>).
<code>400 invalid amount</code>	Amount missing / not positive.
<code>409</code>	This account can't take live payments yet (no payment gateway or receiving UPI ID is set up). Contact your Katana account manager.
<code>502</code>	The payment gateway refused the order — the message gives its reason (for example, below its minimum amount). No order was created; fix and retry with the same <code>txnid</code> .
<code>422</code>	The order is outside your account's limits. No order was created. The body carries <code>code</code> , <code>field</code> , <code>limit</code> and <code>actual</code> — see below.
<code>429 RATE_LIMITED</code>	Too many orders in one second. Send it again after the <code>Retry-After</code> header (1 second).

Limits. A live order outside your account's limits is refused with `422` :

```
{ "error": "amount is above the maximum of ₹5,000", "code": "AMOUNT_ABOVE_MAX", "field":  
"amount", "limit": 5000, "actual": 5000.01 }
```

code	Meaning
<code>AMOUNT_BELOW_MIN</code>	The amount is under the minimum for one order.
<code>AMOUNT_ABOVE_MAX</code>	The amount is over your account's maximum for one order.
<code>UPI_LIMIT_EXCEEDED</code>	The amount is over what one UPI payment can carry (₹1,00,000 unless your account has a higher maximum).

Today's orders plus this one would pass your daily limit; `actual` is the total it would reach. The day is the calendar day in India, and an order that fails or expires gives its amount back.

Request id. Every response carries an `X-Request-Id` header. Send your own (8 to 64 letters, digits, `.`, `_`, `:` or `-`) and it is echoed and kept with the order; otherwise one is generated. Quote it when you contact support.

Compute `hash` with the **scheme configured on your account** (shown next to your Key in the dashboard), then send it as the `hash` field. You never send the scheme name — Katana already knows which one your account uses.

```
message = txnid + "|" + amount + "|" + productinfo + "|" + email
hash     = HMAC_SHA256( key = KEY + SALT , message )      // lowercase hex
```

Only for pairs issued before HMAC_SHA256 became the only scheme; a regenerated pair is always HMAC_SHA256. Used by accounts migrated from an older checkout. Pipe-joined, with five user-defined fields and five reserved blanks before the salt:

[illegible]

Send the customer's browser to the `pay_url` from the response. The hosted page shows a UPI QR plus app buttons and live status (or, for a merchant paid on a gateway's page, the order and a Pay button that opens it). When the payment is confirmed it automatically redirects back to your `return_url` with query params:

https://your-site.com/return?order_id=ORDER-1001&status=SUCCESS&rrn=123456789012

Always re-confirm server-side (callback or status enquiry) before fulfilling — the browser redirect is **informational only** and can be forged by anyone who can type a URL.

6. Receive the status callback

On every terminal status Katana **POSTs** a JSON body to your `notify_url` (or your saved webhook URL). Reply `HTTP 200`. Delivery is queued and **retried with backoff** until you do, and every attempt is recorded — so a callback is never silently dropped.

Callback body

```
{
  "PAY_ID": "...", "ORDER_ID": "ORDER-1001", "TXN_ID": "...",
  "AMOUNT": "100", "CURRENCY_CODE": "356",
  "STATUS": "Captured", "RESPONSE_CODE": "000",
  "RRN": "123456789012", "RESPONSE_DATE_TIME": "2026-07-01T...Z",
  "HASH": "716346F5...EC21"
}
```

One sample for each outcome

The body has the same fields whatever happened. These are the three statuses a callback can carry, and the second callback of an order that is paid late:

```
// Paid
{ "ORDER_ID": "ORDER-1001", "STATUS": "Captured", "RESPONSE_CODE": "000", "RRN":
"123456789012", ... }

// Failed: the payment was declined. RRN is an empty string.
{ "ORDER_ID": "ORDER-1002", "STATUS": "Failed", "RESPONSE_CODE": "004", "RRN": "", ... }

// Expired: no payment arrived in time. RRN is an empty string.
{ "ORDER_ID": "ORDER-1003", "STATUS": "Expired", "RESPONSE_CODE": "003", "RRN": "", ... }

// The same order, paid after it expired: a SECOND callback, some time later
{ "ORDER_ID": "ORDER-1003", "STATUS": "Captured", "RESPONSE_CODE": "000", "RRN":
"123456789013", ... }
```

In the portal, **Orders** finds any order by its order id, your `txnid` or the bank reference and shows every callback attempt with what your server answered; **Resend** sends one again, and **Send test event** sends a sample of each status to your callback URL.

Verify the HASH (with your Salt)

1. Take every field **except** `HASH` .
2. Sort the keys **ascending**; join as `KEY=value` with `~` between pairs.
3. Append your `SALT` directly to the end of that string.
4. `SHA256` it → hex → **UPPERCASE**. It must equal `HASH` .

Paid = `STATUS` is `Captured` and `RESPONSE_CODE` is `000` .

Test callbacks. A callback for a test order also carries `"LIVEMODE": "false"` (included in the HASH like any other field) and is signed with the **test Salt**. Live callbacks omit the field. Verify with the Salt of the matching mode, and never fulfil a real order from a `LIVEMODE=false` callback.

Treat callbacks as idempotent. The same `ORDER_ID` may arrive more than once (a retry). Key your handler on `ORDER_ID` + `STATUS` and make a repeat of the same status a no-op.

Expired and Failed can be followed by Captured . If the customer's payment goes through after the order expired, or after an earlier attempt failed, Katana honours it: the order becomes `SUCCESS` and a **second callback** with `STATUS = "Captured"` is sent. Let `Captured` overwrite an earlier `Expired` or `Failed` . `Captured` is final — nothing is ever sent after it.

7. Status enquiry (and long-poll)

At any time: `GET /api/pay-status/{order_id}`

```
{ "order_id": "ORDER-1001", "amount": 100, "currency_code": "INR",  
  "status": "SUCCESS", "terminal": true, "rrn": "123456789012",  
  "mode": "QR", "livemode": true, "return_url": "..." }
```

`terminal: true` means the order has reached a final state and will not change again.

Long-poll: `?wait=1`

Add `?wait=1` and the request is **held open until the order becomes terminal** (up to ~25s), instead of returning the current state immediately. This lets your own page flip to "Payment received" within about half a second of the money landing, without polling in a tight loop.

```
GET https://katanapay.co/api/pay-status/8d20c0b6-...?wait=1
```

Use plain enquiry (no `wait`) for reconciliation sweeps, and `wait=1` only where a person is watching a screen. Either way, enquiry is the **fallback if a callback was missed** — it always reflects the truth.

Pay-in report

POST `/api/v1/reports/payins` returns your own orders for a date range, with totals.

Field	Meaning
<code>key</code>	Your Key. A test Key reports test orders, a live Key live ones.
<code>from</code> , <code>to</code>	Calendar days in India, <code>YYYY-MM-DD</code> . At most 31 days.
<code>format</code>	<code>json</code> (default) or <code>csv</code> .
<code>hash</code>	Signed like an order, over the string <code>from to</code> .

The JSON carries `summary` (orders, paid orders, paid amount, failed, expired, pending, success rate), `by_day` , `by_flow` and `orders` (your `txnid` , the order `id` , `amount` , `status` , `flow` , `utr` , `created_at` , `paid_at`). `status` is the same word your callback carries: `Captured` , `Failed` , `Expired` , or `Pending` . A report holds up to 10,000 orders; `truncated: true` means the range has more, and the totals still cover all of them.

Every report carries an `X-Report-Hash` header, the SHA-256 of its content (the JSON repeats it as `report_hash`). Keep it with the file: a copy that has been edited no longer matches.

8. Status & response codes

STATUS	RESPONSE_CODE	Terminal	Meaning
<code>Captured</code>	000	yes	Paid — fulfil the order.
<code>Pending</code>	005	no	Awaiting payment.
<code>Failed</code>	004	yes *	Declined / failed.
<code>Expired</code>	003	yes *	Payment request lapsed.

* `Failed` and `Expired` end the order unless the payment goes through afterwards; a `Captured` callback then follows (see §6). `Captured` never changes.

9. Traceability — following one payment end to end

Every pay-in carries four identifiers. Store all of them against your order; each answers a different question.

Identifier	Who creates it	Use it to
<code>txnid / ORDER_ID</code>	You , on create	Your own reference. Idempotency key on create, and the key you match callbacks on.
<code>order.id</code> (UUID)	Katana	Address the hosted page (<code>/pay/{id}</code>) and the status endpoint.
<code>PAY_ID</code>	Katana	Quote to support — it identifies the payment inside Katana.
<code>RRN / UTR</code>	The UPI network	The 12-digit bank reference. This is what a bank statement shows, so it is the identifier that settles disputes.

The lifecycle

```
create order      → status PENDING      (you hold txnid + order.id)
customer pays by UPI → credit confirmed
                  → status SUCCESS, RRN filled in
                  → signed callback POSTed to your notify_url (retried until 200)
                  → customer redirected to your return_url
```

A payment that never completes ends **Expired**; one that fails ends **Failed**. Both are terminal and both fire a callback, so your side never has to guess. If the money arrives afterwards all the same, the order moves to **SUCCESS** and a `Captured` callback follows.

Reconciling a day

- For any order, `GET /api/pay-status/{order_id}` is the authoritative answer at that moment.
- Match your ledger to the bank's on **RRN** — it is the only identifier both sides share.
- In the dashboard, **Transactions** lists every order for your account with its status, RRN and timestamps, and exports the same view as CSV for the period you filter to.
- If a callback never arrived, enquiry will still show the true status — a missed callback never means a lost payment.

Reconciliation rule of thumb: your system should be able to answer "what happened to `ORDER-1001` ?" from your own database alone. That is what storing the four identifiers above buys you.

10. Sample code

PHP — create order

```
$key="mk_xxx"; $salt="your_salt";
$o = ["txnid"=>"ORDER-1001","amount"=>"100","productinfo"=>"Order 1001","email"=>"d@e.com"];
$msg = $o["txnid"]."|" . $o["amount"]."|" . $o["productinfo"]."|" . $o["email"];
$o["hash"] = hash_hmac("sha256", $msg, $key.$salt);
$o["key"]=$key; $o["return_url"]="https://you.com/return";
$o["notify_url"]="https://you.com/callback";
$ch=curl_init("https://katanapay.co/api/v1/katana-pay/order");
curl_setopt_array($ch, [CURLOPT_POST=>1,CURLOPT_RETURNTRANSFER=>1,
    CURLOPT_HTTPHEADER=>["Content-Type:
application/json"],CURLOPT_POSTFIELDS=>json_encode($o)]);
$res=json_decode(curl_exec($ch),true);
header("Location: ".$res["pay_url"]);    // redirect customer
```

Node.js — create order

```
import crypto from "crypto";
const key="mk_xxx", salt="your_salt";
const o={ txnid:"ORDER-1001", amount:"100", productinfo:"Order 1001", email:"d@e.com" };
const msg=[o.txnid,o.amount,o.productinfo,o.email].join("|");
o.hash=crypto.createHmac("sha256",key+salt).update(msg).digest("hex");
const r=await fetch("https://katanapay.co/api/v1/katana-pay/order",{method:"POST",
    headers:{"Content-Type":"application/json"},

body:JSON.stringify({...o,key,return_url:"https://you.com/return",notify_url:"https://you.com/
callback"}}));
const data=await r.json();    // redirect customer to data.pay_url
```

Python — create order

```
import hmac,hashlib,requests
key,salt="mk_xxx","your_salt"
o={"txnid":"ORDER-1001","amount":"100","productinfo":"Order 1001","email":"d@e.com"}
msg="|".join([o["txnid"],o["amount"],o["productinfo"],o["email"]])
o["hash"]=hmac.new((key+salt).encode(),msg.encode(),hashlib.sha256).hexdigest()
o.update(key=key, return_url="https://you.com/return", notify_url="https://you.com/callback")
data=requests.post("https://katanapay.co/api/v1/katana-pay/order",json=o).json()
# redirect customer to data["pay_url"]
```

Verify a callback (Node.js)

```
function verify(body, salt){
  const got=body.HASH; const f={...body}; delete f.HASH;
  const str=Object.keys(f).sort().map(k=>`${k}=${f[k]==null?"":f[k]}`).join("~")+salt;
  const exp=crypto.createHash("sha256").update(str,"utf8").digest("hex").toUpperCase();
  return exp===got;    // true → trust it; then check STATUS=="Captured"
}
```

Verify a callback (PHP)

```
function verify($body,$salt){
  $got=$body["HASH"]; unset($body["HASH"]); ksort($body);
  $parts=[]; foreach($body as $k=>$v){$parts[]=$k."($v??"");}
  $str=implode("~",$parts).$salt;
  return strtoupper(hash("sha256",$str))===$got;
}
```

11. Test mode

Integrate with the **test pair** first. Test orders behave exactly like live ones (hosted page, callbacks, status enquiry) except that they pay a sandbox UPI ID and the hosted page shows a **Test payment** label with **Simulate success** and **Simulate failure** buttons. Tap one and the order settles that way; your server receives the same signed callback a real payment sends. From your own code, the pause of the amount also forces the outcome, so every branch can be tested without a real payer:

Send amount	Order becomes	Use it to test
100.99	SUCCESS after ~8 s	Happy path, callback verification, fulfilment
100.13	FAILED	Decline handling
100.11	EXPIRED	Expiry handling
100.00	stays PENDING , expires after 15 min	Long-poll and your own re-check

These rules apply **only** to test orders. A live order changes state only on a real confirmed credit, and an amount ending in .99 behaves exactly like any other. Real bank credits never confirm a test order.

In the dashboard, the **Test / Live** switch in the header shows test orders; an orange banner stays on while you are viewing test data. Live totals, statements and settlement never include test orders.

12. Go-live checklist

Live mode must be activated first. Live keys and live orders are locked until Katana activates live mode for your account. In the dashboard, open **Integration** → **Activate live mode**: once every item is ticked (onboarding approved, settlement UPI ID saved, webhook URL set, one successful test payment), tap **Request activation**. Until it is approved a live Key is refused with `403` and `"code": "LIVE_MODE_NOT_ACTIVATED"`.

- Complete a test integration with the **test Key + Salt**: success, failure and expiry callbacks all verify (§11).
- Request **live mode activation** and wait for approval.
- Generate your **live Key + Salt**, store the Salt securely (server-side only) and swap both on your server.
- Set your default **return_url** and **webhook URL** in the dashboard (or pass them per order).
- Store `txnid`, `order.id`, `PAY_ID` and `RRN` against your order (§9).
- Make a small live payment; confirm you receive the callback and the **HASH verifies** with the live Salt.
- Handle callbacks **idempotently** — the same `ORDER_ID` may arrive more than once.
- Make create-order **retry-safe** by always reusing the same `txnid`.
- Always confirm **server-side** before fulfilling — never trust the browser redirect alone.

Katana Pay · integration support: [contact your Katana account manager](#).